
Session 2 (2h) — GPIO (General Purpose Input/Output)

Covers: **GPIO**, continues 아두이노 회로 구성 (on-site task)

1. Digital output — ~25 min

Already touched in Session 1. Quick review + extend to multiple pins:

```
const int leds[] = {6, 7, 8};
for (int i = 0; i < 3; i++) pinMode(leds[i], OUTPUT);
```

Useful when a real task needs several outputs (multiple LEDs/relays/actuators).

2. Digital input — ~45 min

- `pinMode(pin, INPUT)` on a switch with nothing else connected leaves the pin **floating** — it reads unpredictable noise.
- `pinMode(pin, INPUT_PULLUP)` enables the MCU's internal pull-up resistor, holding the pin HIGH by default. Wire the switch between the pin and GND. **Logic is inverted: pressed = LOW, released = HIGH.**
- Reading it: `int state = digitalRead(pin);`

3. GPIO as the foundation for everything else — ~20 min

Every sensor/actuator interaction in the internship (아두이노 회로 구성, 센서 연동) starts as a GPIO read or write. ADC (Session 3) and interrupts (Session 4) are both built on top of the same physical pins — you're just using them in more advanced modes.

4. Detecting a state *change*, not just a level — ~30 min

Reading “is it pressed right now” every loop iteration causes flicker if you try to toggle something. You need to detect the edge (moment of change):

```
if (buttonState == LOW && lastButtonState == HIGH) {
    // this is the instant it was just pressed
}
```

Practice: `practice/button_led_gpio.ino` — ~30 min

Button (D2, INPUT_PULLUP) toggles an LED (D8) on/off — one full press-and-release cycle = one toggle, no flicker.

Wiring for this session

Component	Pin	Other end
LED (+)	D8	220Ω resistor → anode
LED (-)	GND	cathode
Switch leg A	D2	—
Switch leg B	GND	—

Checkpoint before Session 3

- ☐ Can explain INPUT vs INPUT_PULLUP and why pressed = LOW
- ☐ LED toggles cleanly on each button press
- ☐ Understand why GPIO is the base layer under ADC/PWM and interrupts